

Digital Design and Computer Organisation Laboratory

3rd Semester, Academic Year 2025

Date:10-Nov-2025

Name	SRN	Section C
DEVOPAM PAL	PES2UG24CS152	
KARAN VARSHNEY	PES2UG24CS903	

MINI PROJECT



**Project : Verilog Morse Code Encoder with
VPI**

1. Project Goal

This report explains our project that turns text typed by a user into Morse code.

The main goal is to show how to connect Verilog (for hardware logic) with C code (for user input). This connection, called VPI (*Verilog Procedural Interface*), lets the Verilog code read text directly from a user's console, which Verilog normally can't do.

2. How It Works

The system uses three main parts that work together:

1. **Verilog Encoder** (*morse_encoder*): The main Verilog logic. It knows how to turn one letter (like "A") into the correct dots and dashes (".-").
2. **Verilog Testbench** (*testbench_encoder_vpi*): A Verilog file that sets up the test. It provides a clock and tells the encoder when to start working.
3. **C Code** (*morse_combined_vpi.c*): A C program that Verilog can call. Its job is to read the user's text from the console.

Here is the step-by-step process:



1. The Verilog simulation starts and resets the encoder.
2. The testbench asks the C code for a character by calling *\$get_next_char*.
3. The C code pauses the simulation and waits for the user to type a line of text and press Enter.
4. The C code gives the *first* character of that line to the testbench.
5. The testbench gives this character to the encoder and tells it to '*start*'.
6. The encoder gets busy, sending out the dots and dashes one by one.
7. The testbench sees these dots and dashes and prints them to the console.
8. When the encoder is finished, the testbench asks the C code for the *next* character from the line.
9. This repeats until the C code runs out of characters (hits the end of the line), and the simulation stops.

3. The Files Explained

3.1. The Encoder (*morse.v*)

- **Module:** *morse_encoder*
- **Purpose:** This is the main design. It's a state machine that turns one letter into the right Morse code symbols.
- **Inputs:** *clk*, *reset*, *ascii_char_in* (the letter to encode), *start*.
- **Outputs:** *encoder_busy* (a "busy" light), *morse_symbol* (the dot/dash), *symbol_ready* (blinks when a new symbol is ready).
- **Functionality:**

- It waits in an *S_IDLE* state.
- When *start* is pulsed, it looks up the letter in a big *case* statement (like a dictionary).
- It finds the pattern (e.g., "A" is dot, then dash).
- It steps through states to send each symbol (., then -) one at a time.
- When done, it goes back to *S_IDLE*.

3.2. The Testbench (*morse_tb.v*)

- **Module:** *testbench_encoder_vpi*
- **Purpose:** To test the *morse_encoder* logic.
- **Key Parts:**
 - **DUT Instantiation:** Connects the testbench to the encoder.
 - **Clock & Reset:** Provides the main clock and the initial reset signal.
 - **Main Control (*initial* block):**
 - Calls *\$get_next_char* to get a letter from C.
 - Loops as long as it gets valid characters.
 - Pulses *start_encode* to make the encoder work.
 - Waits for the encoder to finish (*@(negedge encoder_busy)*).
 - **Output Monitor (*always* block):**
 - Watches for the *symbol_ready* blink.
 - When it sees it, it prints the *morse_symbol* (as . or -) to the screen.

3.3. The C Input Helper (*morse_combined_vpi.c*)

- **Purpose:** To give Verilog the custom command *\$get_next_char*.
- **Key Functions:**
 - *get_next_char_calltf*: This is the C code for *\$get_next_char*.
 - The first time it's called, it pauses Verilog and waits for the user to type a line (*fgets()*).
 - It stores this line in a buffer.
 - It gives the first character back to Verilog.
 - Every time it's called after that, it just gives the *next* character from the buffer.
 - It also converts all letters to uppercase (*toupper()*).
 - When the line is finished, it returns 0 to tell Verilog to stop.

4. How to Run This Project

This project needs to be built in two stages. The following commands are for a Linux/macOS environment using the Icarus Verilog toolchain.

1. Compilation:

First, you must compile the C code into a VPI library. Then, you compile the Verilog code and link it to that library.

Step 1: Compile the C code into a .vpi file

This command tells the C compiler to build a shared library

```
gcc -shared -fPIC -o morse_combined.vpi morse_combined_vpi.c -I/usr/include/iverilog/
```

```
# Step 2: Compile the Verilog code
# -m morse_combined tells iverilog to load our VPI module
# -o my_vpi_encoder specifies the output simulation file
iverilog -m morse_combined.vpi -o my_vpi_encoder morse.v morse_tb.v
```

2. Execution:

Once compiled, you run the simulation file using vvp.

```
# Step 3: Run the compiled simulation
vvp my_vpi_encoder
```

- When you run this, the simulation will start and then pause, waiting for you to type.
- Type a message (e.g., "HELLO") and press Enter.
- The simulation will unpauses and process your text, printing the Morse code output.

5. File Summary

- *morse.v*: The Verilog encoder logic (the design).
- *morse_tb.v*: The Verilog file that tests the design.
- *morse_combined_vpi.c*: The C code that lets Verilog read user input.

6. Output

```
PS C:\Users\Admin\Desktop\MINI_PROJECT\karan's> gcc -shared -o morse_combined.vpi morse_combined_vpi.c -IC:/iverilog/include/iverilog C:/iverilog/lib/libvpi.a
PS C:\Users\Admin\Desktop\MINI_PROJECT\karan's> iverilog -o my_vpi_encoder morse.v morse_tb.v
PS C:\Users\Admin\Desktop\MINI_PROJECT\karan's> vvp -M. -m morse_combined my_vpi_encoder
Testbench starting...
Reset released. Waiting for first input...
SOS DEV KARAN
MORSE CODE : ... --- ... -.. . .... -.- .- .- .- .-

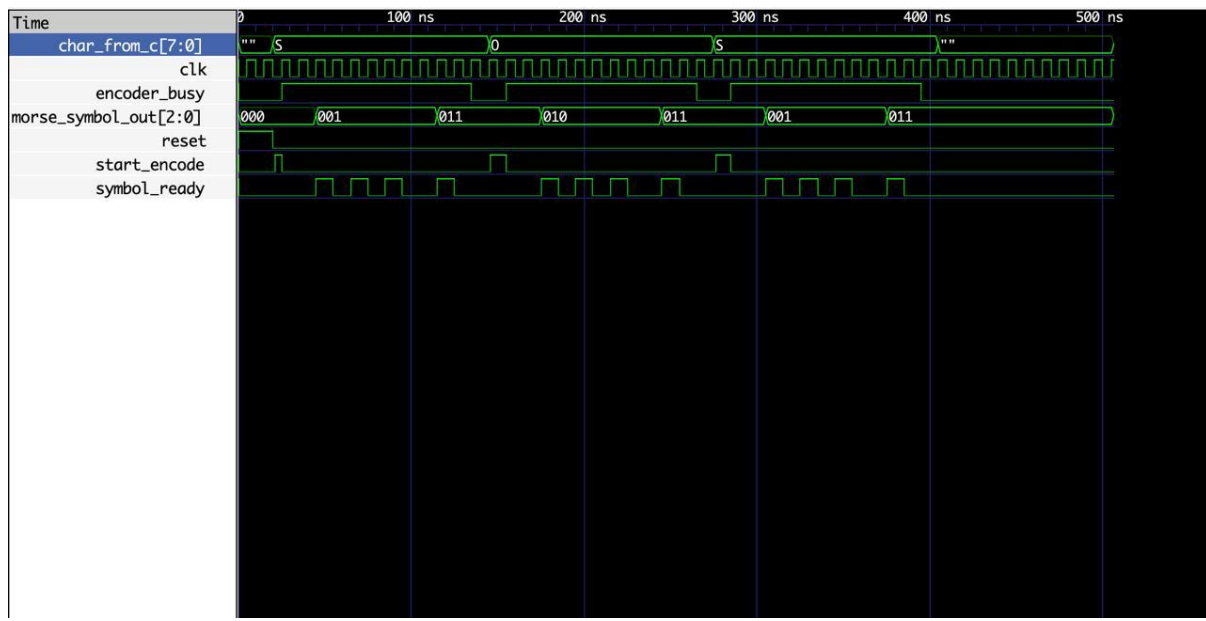
PRECODE: SOS DEV KARAN
End of input (ASCII 0) received. Finishing.
PS C:\Users\Admin\Desktop\MINI_PROJECT\karan's> |
```

```
PS C:\Users\Admin\Desktop\MINI_PROJECT\karan's> vvp -M. -m morse_combined my_vpi_encoder
Testbench starting...
Reset released. Waiting for first input...
A B C
MORSE CODE : .- -... -.-.

PRECODE: A B C
End of input (ASCII 0) received. Finishing.
PS C:\Users\Admin\Desktop\MINI_PROJECT\karan's> vvp -M. -m morse_combined my_vpi_encoder
Testbench starting...
Reset released. Waiting for first input...
ABC
MORSE CODE : .- -... -.-.

PRECODE: ABC
End of input (ASCII 0) received. Finishing.
PS C:\Users\Admin\Desktop\MINI_PROJECT\karan's> |
```

GTKWAVE OUTPUT:



CODES :

Morse.v:



```
`timescale 1ns / 1ps

/*
 * This is your Morse Code Encoder module (DUT).
 * It takes one ASCII character and a 'start' pulse.
 * It outputs the morse symbols one by one.
 */
module morse_encoder (
    input wire clk,
    input wire reset,

    // --- Control inputs from testbench ---
    input wire [7:0] ascii_char_in, // The character to encode
    input wire start,               // Pulse to start encoding

    // --- Status outputs ---
    output reg encoder_busy,        // High while encoding

    // --- Morse symbol output ---
    output reg [2:0] morse_symbol, // 1=dot, 2=dash, 3=end-of-char, 4=end-of-
word
```

```

output reg symbol_ready      // Pulse high for one cycle with each symbol
);
reg [7:0] char_to_encode;
reg [3:0] state;
reg [4:0] symbol_index; // Index for which symbol we are sending
reg [2:0] morse_pattern [0:7];
reg [4:0] morse_len; // How many symbols for this char

// --- State definitions ---
localparam S_IDLE = 0;
localparam S_LOOKUP = 1;
localparam S_SEND_SYMBOL = 2;
localparam S_WAIT_FOR_NEXT_SYMBOL = 3;
localparam S_SEND_GAP = 4;
localparam S_WAIT_AFTER_GAP = 5;

always @(posedge clk or posedge reset) begin
    if (reset) begin
        encoder_busy <= 0;
        symbol_ready <= 0;
        morse_symbol <= 0;
        char_to_encode <= 0;
        state <= S_IDLE;
        symbol_index <= 0;
    end else begin
        // Defaults
        symbol_ready <= 0;

        case (state)
            S_IDLE: begin
                encoder_busy <= 0;
                if (start) begin
                    char_to_encode <= ascii_char_in;
                    state <= S_LOOKUP;
                    encoder_busy <= 1;
                    symbol_index <= 0;
                end
            end

            S_LOOKUP: begin
                // ... (lookup table is unchanged) ...
                case (char_to_encode)
                    "A": begin morse_pattern[0]=1; morse_pattern[1]=2;
morse_len=2; end // .-
                    "B": begin morse_pattern[0]=2; morse_pattern[1]=1;
morse_pattern[2]=1; morse_pattern[3]=1; morse_len=4; end // -...
                    "C": begin morse_pattern[0]=2; morse_pattern[1]=1;
morse_pattern[2]=2; morse_pattern[3]=1; morse_len=4; end // -.-.

```

```

        "D": begin morse_pattern[0]=2; morse_pattern[1]=1;
morse_pattern[2]=1; morse_len=3; end // -..
        "E": begin morse_pattern[0]=1; morse_len=1; end // .
        "F": begin morse_pattern[0]=1; morse_pattern[1]=1;
morse_pattern[2]=2; morse_pattern[3]=1; morse_len=4; end // ...-
        "G": begin morse_pattern[0]=2; morse_pattern[1]=2;
morse_pattern[2]=1; morse_len=3; end // --.
        "H": begin morse_pattern[0]=1; morse_pattern[1]=1;
morse_pattern[2]=1; morse_pattern[3]=1; morse_len=4; end // ....
        "I": begin morse_pattern[0]=1; morse_pattern[1]=1;
morse_len=2; end // ..
        "J": begin morse_pattern[0]=1; morse_pattern[1]=2;
morse_pattern[2]=2; morse_pattern[3]=2; morse_len=4; end // .---
        "K": begin morse_pattern[0]=2; morse_pattern[1]=1;
morse_pattern[2]=2; morse_len=3; end // -.-
        "L": begin morse_pattern[0]=1; morse_pattern[1]=2;
morse_pattern[2]=1; morse_pattern[3]=1; morse_len=4; end // -...
        "M": begin morse_pattern[0]=2; morse_pattern[1]=2;
morse_len=2; end // --
        "N": begin morse_pattern[0]=2; morse_pattern[1]=1;
morse_len=2; end // -..
        "O": begin morse_pattern[0]=2; morse_pattern[1]=2;
morse_pattern[2]=2; morse_len=3; end // ---
        "P": begin morse_pattern[0]=1; morse_pattern[1]=2;
morse_pattern[2]=2; morse_pattern[3]=1; morse_len=4; end // ---.
        "Q": begin morse_pattern[0]=2; morse_pattern[1]=2;
morse_pattern[2]=1; morse_pattern[3]=2; morse_len=4; end // ---.
        "R": begin morse_pattern[0]=1; morse_pattern[1]=2;
morse_pattern[2]=1; morse_len=3; end // -.-
        "S": begin morse_pattern[0]=1; morse_pattern[1]=1;
morse_pattern[2]=1; morse_len=3; end // ...
        "T": begin morse_pattern[0]=2; morse_len=1; end // -
        "U": begin morse_pattern[0]=1; morse_pattern[1]=1;
morse_pattern[2]=2; morse_len=3; end // ...-
        "V": begin morse_pattern[0]=1; morse_pattern[1]=1;
morse_pattern[2]=1; morse_pattern[3]=2; morse_len=4; end // ....-
        "W": begin morse_pattern[0]=1; morse_pattern[1]=2;
morse_pattern[2]=2; morse_len=3; end // --.
        "X": begin morse_pattern[0]=2; morse_pattern[1]=1;
morse_pattern[2]=1; morse_pattern[3]=2; morse_len=4; end // -...-
        "Y": begin morse_pattern[0]=2; morse_pattern[1]=1;
morse_pattern[2]=2; morse_pattern[3]=2; morse_len=4; end // -...-
        "Z": begin morse_pattern[0]=2; morse_pattern[1]=2;
morse_pattern[2]=1; morse_pattern[3]=1; morse_len=4; end // ---..
        " ": begin morse_symbol <= 4; symbol_ready <= 1;
morse_len = 0; state <= S_IDLE; end // 4 = end-of-word
        default: begin morse_len <= 0; state <= S_IDLE; end //
Unknown char, just go back to idle

```

```

        endcase

        if (char_to_encode != " ") begin
            state <= S_SEND_SYMBOL;
        end
    end

    S_SEND_SYMBOL: begin
        if (symbol_index < morse_len) begin
            morse_symbol <= morse_pattern[symbol_index];
            symbol_ready <= 1;
            symbol_index <= symbol_index + 1;
            state <= S_WAIT_FOR_NEXT_SYMBOL;
        end else begin
            // Done with dots/dashes, send end-of-char gap
            state <= S_SEND_GAP;
        end
    end

    // this state to create a 1-cycle pause
    // between symbols. This ensures symbol_ready is a clean
    // 1-cycle pulse for each symbol.
    S_WAIT_FOR_NEXT_SYMBOL: begin
        // symbol_ready is low by default
        state <= S_SEND_SYMBOL; // Go back to check for next
symbol

    end

    S_SEND_GAP: begin
        morse_symbol <= 3; // 3 = end of character
        symbol_ready <= 1;
        state <= S_WAIT_AFTER_GAP;
    end

    // after the end-of-char gap. This ensures encoder_busy
    // doesn't go low on the same cycle as the last symbol_ready.
    S_WAIT_AFTER_GAP: begin
        // symbol_ready is low by default (set at top)
        state <= S_IDLE; // Now go to idle
    end

endcase
end
end

endmodule

```

morse_tb.v


```

`timescale 1ns / 1ps

module testbench_encoder_vpi;

    // --- Testbench internal signals ---
    reg clk;
    reg reset;

    // --- Signals to link C code and DUT ---
    reg [7:0] char_from_c; // C code writes to this
    reg start_encode;      // Pulse to start the encoder

    // --- Wires from DUT ---
    wire encoder_busy;
    wire [2:0] morse_symbol_out;
    wire symbol_ready;

    // --- Instantiate the Design Under Test (DUT) ---
    morse_encoder dut (
        .clk(clk),
        .reset(reset),
        .ascii_char_in(char_from_c),
        .start(start_encode),
        .encoder_busy(encoder_busy),
        .morse_symbol(morse_symbol_out),
        .symbol_ready(symbol_ready)
    );

    // --- Main Simulation Block (State Machine) ---
    initial begin
        $display("Testbench starting...");

        // 1. Initialize signals and reset the DUT
        clk = 0;
        reset = 1;
        char_from_c = 0;
        start_encode = 0;

        #20;
        reset = 0; // Release the reset
        $display("Reset released. Waiting for first input...");

        // 2. Call C to get the first character
        $get_next_char(char_from_c);

        // Add a small delay after the VPI call to let the
        // value settle before starting the loop.
        #1;
    end
endmodule

```

```

// 3. Loop until C code gives ASCII 0 (end of input)
while (char_from_c != 0) begin

    // --- FIXED LOGIC ---
    // 1. Set start high
    start_encode <= 1;

    // 2. Wait for the DUT to see it on the next clock edge
    @(posedge clk);
    // On this edge, the DUT sees start=1 and sets busy=1

    // 3. Now set start low
    start_encode <= 0;

    // 4. Wait for the encoder to finish
    @(negedge encoder_busy);

    // 5. Wait one more cycle for safety before getting next char
    @(posedge clk);

    // Get the next character from C
    $get_next_char(char_from_c);
    #1;
end

// 4. End of simulation
$display("End of input (ASCII 0) received. Finishing.");

#100; // Wait a bit
$finish; // Stop the simulation
end

// --- Clock Generator ---
always #5 clk = ~clk;

// --- Output Monitor ---
// This block prints the morse code as it is generated
always @(posedge clk) begin
    if (symbol_ready) begin
        case (morse_symbol_out)
            1: $write(".");
            2: $write("-");
            3: $write(" "); // End of char
            4: $write(" "); // End of word (was "/" ")
        endcase
        $fflush; // Flush stdout to make sure it prints immediately
    end
end

```

```

        end
    end

endmodule

```

morse_combined_vpi.c

```

#include <stdio.h>
#include <string.h>
#include <stdlib.h>
#include <ctype.h>
#include "vpi_user.h"

//=====
// Logic from vpi_encoder_helper.c
//=====

static char input_buffer[1024];
static int buffer_index = -1;

PLI_INT32 get_next_char_calltf(PLI_BYTE8 *user_data) {
    (void)user_data;
    vpiHandle systf_handle, arg_iterator, arg_handle;
    s_vpi_value value_s;
    PLI_INT32 ascii_char = 0;

    if (buffer_index == -1) {
        // We no longer print a prompt, just wait for input.
        if (fgets(input_buffer, sizeof(input_buffer), stdin) == NULL) {
            input_buffer[0] = '\0';
        } else {
            // After getting input, print the header for the morse code.
            vpi_printf("MORSE CODE : ");
            fflush(stdout);
        }
        buffer_index = 0;
    }

    char c = input_buffer[buffer_index];

    if (c == '\n' || c == '\0') {
        ascii_char = 0;
        buffer_index = -1;

        // The Verilog side has finished printing the morse code for the line.
        // Now, we print the original text in the "PRECODE" format.
    }
}

```

```

        printf("\n\nPRECODE: ");
        // Remove the trailing newline from the buffer before printing.
        input_buffer[strcspn(input_buffer, "\r\n")] = 0;
        printf("%s\n", input_buffer);
        fflush(stdout);

    } else {
        c = toupper(c);
        ascii_char = (PLI_INT32)c;
        buffer_index++;
    }

    systf_handle = vpi_handle(vpiSysTfCall, NULL);
    arg_iterator = vpi_iterate(vpiArgument, systf_handle);
    arg_handle = vpi_scan(arg_iterator);

    if (!arg_handle) {
        vpi_printf("ERROR: $get_next_char requires 1 argument.\n");
        vpi_free_object(arg_iterator);
        return 0;
    }

    value_s.format = vpiIntVal;
    value_s.value.integer = ascii_char;
    vpi_put_value(arg_handle, &value_s, NULL, vpiNoDelay);
    vpi_free_object(arg_iterator);

    return 0;
}

void register_get_next_char_task(void) {
    s_vpi_systf_data tf_data;
    tf_data.type = vpiSysTask;
    tf_data.sysfunctype = 0;
    tf_data.tfname = "$get_next_char";
    tf_data.calltf = get_next_char_calltf;
    tf_data.compiletf = NULL;
    tf_data.sizetf = NULL;
    tf_data.user_data = NULL;
    vpi_register_systf(&tf_data);
}

//=====
// Logic from morse_vpi.c
//=====

void print_morse_logic(int pattern, int len) {
    if (len == 0) {

```

```

        printf(" ");
        return;
    }

    for (int i = 0; i < len; ++i) {
        int symbol = (pattern >> ((len - 1 - i) * 2)) & 0x3;
        if (symbol == 0x1) {
            printf(".");
        } else if (symbol == 0x2) {
            printf("-");
        }
    }
    printf(" ");
}

static int print_morse_vpi(char *user_data) {
    (void)user_data;
    vpiHandle systf_handle = vpi_handle(vpiSysTfCall, NULL);
    vpiHandle arg_itr = vpi_iterate(vpiArgument, systf_handle);
    vpiHandle pattern_handle = vpi_scan(arg_itr);
    vpiHandle len_handle = vpi_scan(arg_itr);

    s_vpi_value val_s;
    val_s.format = vpiIntVal;

    vpi_get_value(pattern_handle, &val_s);
    int pattern = val_s.value.integer;

    vpi_get_value(len_handle, &val_s);
    int len = val_s.value.integer;

    print_morse_logic(pattern, len);
    fflush(stdout);

    vpi_free_object(arg_itr);
    return 0;
}

void register_print_morse_task() {
    s_vpi_systf_data tf_data;
    tf_data.type = vpiSysTask;
    tf_data.tfname = "$print_morse";
    tf_data.calltf = print_morse_vpi;
    tf_data.compiletf = NULL;
    tf_data.sizetf = NULL;
    tf_data.user_data = NULL;
    vpi_register_systf(&tf_data);
}

```

```
//=====
// Entry point for Verilog
//=====

void (*vlog_startup_routines[])() = {
    register_get_next_char_task,
    register_print_morse_task,
    0};
```

